

16. SQLException in JDBC

Exception handling allows you to handle exceptional conditions such as program-defined errors in a controlled fashion. When an exception condition occurs, an exception is thrown. The term thrown means that current program execution stops, and the control is redirected to the nearest applicable catch clause. If no applicable catch clause exists, then the program's execution ends. JDBC Exception handling is very similar to the Java Exception handling but for JDBC, the most common exception you'll deal with is `java.sql.SQLException`.

SQLException Methods

An `SQLException` can occur both in the driver and the database. When such an exception occurs, an object of type `SQLException` will be passed to the catch clause. The passed `SQLException` object has the following methods available for retrieving additional information about the exception –

Method	Description
<code>getErrorCode()</code>	Gets the error number associated with the exception.
<code>getMessage()</code>	Gets the JDBC driver's error message for an error, handled by the driver or gets the Oracle error number and message for a database error.
<code>getSQLState()</code>	Gets the XOPEN SQLstate string. For a JDBC driver error, no useful information is returned from this method. For a database error, the five-digit XOPEN SQLstate code is returned. This method can return null.
<code>getNextException()</code>	Gets the next Exception object in the exception chain.
<code>printStackTrace()</code>	Prints the current exception, or throwable, and it's backtrace to a standard error stream.
<code>printStackTrace(PrintStream s)</code>	Prints this throwable and its backtrace to the print stream you specify.
<code>printStackTrace(PrintWriter w)</code>	Prints this throwable and it's backtrace to the print writer you specify.

By utilizing the information available from the Exception object, you can catch an exception and continue your program appropriately. Here is the general form of a try block –

```
try {  
    // Your risky code goes between these curly braces!!!  
}  
catch(Exception ex) {
```

```

// Your exception handling code goes between these
// curly braces, similar to the exception clause
// in a PL/SQL block.
}
finally {
    // Your must-always-be-executed code goes between these
    // curly braces. Like closing database connection.
}

```

Facing SQLException for Invalid Table Name and using Error Codes in JDBC Example

In this example, we've three static string for database connection string, username, password. In main method, we're preparing connection to the database using `DriverManager.getConnection()` method. We prepared a Statement using `connection.createStatement()` method. Using `statement.executeQuery()`, a query with invalid table name is executed which results in a `SQLException`. Exception is handled in catch block and error code, sql state and a detailed error message is printed.

```

package com.lokesh;

import java.sql.*;

public class SQLExceptionExample1 {
    static final String MYSQL_URL = "jdbc:mysql://localhost/organization";
    static final String USER_NAME = "root";
    static final String PASSWORD = "root";

    public static void main(String args[]) {
        try {
            Connection conn = DriverManager.getConnection(MYSQL_URL, USER_NAME, PASSWORD);

            Statement stmt = conn.createStatement();
            // Giving incorrect table name to get Exceptions
            ResultSet rs = stmt.executeQuery("SELECT * FROM EMPLOYEES10");

            while( rs.next()){
                System.out.println("ID:" + rs.getInt("id"));
                System.out.println(", Age: " + rs.getInt("age"));
            }

        } catch(SQLException e) {
            int errorCode = e.getErrorCode();
            String sqlState = e.getSQLState();
            String errorMsg = e.getMessage();

            System.out.println("Error code: " + errorCode);
            System.out.println("SqlState: " + sqlState);
            System.out.println("Error Message: " + errorMsg);

            e.printStackTrace();
        }
    }
}

```

Output:

```

java.sql.SQLSyntaxErrorException: Table 'organization.employees10' doesn't exist
Error code: 1146

```

```

SqlState: 42S02
Error Message: Table 'organization.employees10' doesn't exist
    at com.mysql.cj.jdbc.exceptions.SQLError.createSQLException(SQLError.java:121)
    at
com.mysql.cj.jdbc.exceptions.SQLExceptionsMapping.translateException(SQLExceptionsMapping.j
ava:122)
    at com.mysql.cj.jdbc.StatementImpl.executeQuery(StatementImpl.java:1200)
    at com.lokesh.SQLExceptionExample1.main(SQLExceptionExample1.java:16)

```

Facing SQLException for Invalid Column Name and using Error Codes in JDBC Example

In this example, we've three static string for database connection string, username, password. In main method, we're preparing connection to the database using `DriverManager.getConnection()` method. We prepared a Statement using `connection.createStatement()` method. Using `statement.executeQuery()`, a query with invalid column name is executed which results in a `SQLException`. Exception is handled in catch block and error code, sql state and a detailed error message is printed.

```

package com.lokesh;

import java.sql.*;

public class SQLExceptionExample2 {

    static final String MYSQL_URL = "jdbc:mysql://localhost/organization";
    static final String USER_NAME = "root";
    static final String PASSWORD = "root";

    public static void main(String args[]) {
        try {
            Connection conn = DriverManager.getConnection(MYSQL_URL, USER_NAME,
                                                         PASSWORD);

            System.out.println("Connection to db established..");
            String query = "select * from employees";
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(query);
            System.out.println("Successfully executed query.");
            while (rs.next()) {
                System.out.println("Intentionally giving emp_id column name,
                                   when the correct " + "one is: id. Giving this, to see
                                   exception handling.");
                System.out.println("ID: " + rs.getString("emp_id"));
            }
        } catch (SQLException e) {
            int errorCode = e.getErrorCode();
            String sqlState = e.getSQLState();
            String errorMsg = e.getMessage();

            System.out.println("Error code: " + errorCode);
            System.out.println("SqlState: " + sqlState);
            System.out.println("Error Message: " + errorMsg);

            System.out.println("-----");
            e.printStackTrace();
        }
    }
}

```

Output:

```
Connection to db established..
Successfully executed query.
Intentionally giving emp_id column name, when the correct one is: id. Giving this, to see
exception handling.
Error code: 0
SqlState: S0022
Error Message: Column 'emp_id' not found.
-----
java.sql.SQLException: Column 'emp_id' not found.
    at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:130)
    at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:98)
    at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:90)
    at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:64)
    at com.mysql.cj.jdbc.result.ResultSetImpl.findColumn(ResultSetImpl.java:584)
    at com.mysql.cj.jdbc.result.ResultSetImpl.getString(ResultSetImpl.java:896)
    at com.lokesh.SQLExceptionExample2.main(SQLExceptionExample2.java:22)
```